

Model weight distribution to 1,000 hosts

front: what to ask, what to design · say the bound first · naive → tree → swarm

Anthropic questions this is: SD Q3 model weight distribution (1point3acres, 13 discussions, phone and onsite; Tyler read the card verbatim): 500 GB, 10 Gbit/s source, 100–1,000 hosts with 10 Gbit/s NICs, 1–5% die; evolve naive → tree → chunked swarm; follow-ups = bad-network hosts, node dies mid-broadcast, coordinator-free recovery, per-chunk hashes + quarantine, cadence, deployment observability, readiness-gated routing · **hellointerview** "large model file distribution" (11 staff reports) adds incast control + rollback · **Kyle #1** rollout variant adds records with generations, stale reports, 3 regions in 15 min, rollback (own cheat sheet) · **prachub** shared-link-cap + "stream a file to 1,000 hosts fastest" · **Q1 follow-up** faster GPU bring-up = same pipeline. **Phone-screen trap**: the interviewer wants the bound and the swarm, not a tour of every intermediate design.

1 REQUIREMENTS 0–5 min · units, then the bound

- **Ask:** Gbit/s or GB/s? one shared up/down budget or full duplex? one file or shards, immutable, checksummed? object store or a single server? flat network or racks? what is "done": bytes, verified, or serving? how often does a version ship?
- **The system:** copies a published version to every target host · every host verifies every byte · finishes with 1–5% of hosts dying and nobody watching · operators see progress and know when it is done.
- **Technical challenges:** the source link is the bottleneck → fan out inside the DC · verify before trust · failed hosts must not stall or poison · no coordinator on the data path · production traffic survives the copy.
- **The bound, before any design:** $500 \text{ GB} \times 8 = 4,000 \text{ Gbit} \rightarrow 400 \text{ s}$ per host over 10 Gbit/s. Nothing beats it. Shared up/down budget → 800 s .

2 CORE ENTITIES 5–8 min · the tracker is a component

- **Artifact**(version, size, chunk list + per-chunk hash, whole-file hash) — immutable
- **Rollout**(id, version, target set, status, started_at, done_at)
- **Host**(id, rack, state idle → downloading → verified → active, chunk bitmap, bytes_done, heartbeat, health)
- **Tracker** state: chunk → hosts holding a verified copy; host → bitmap + health. Three jobs: who has what, who is healthy, global progress. Never on the data path.
- **Chunk size:** 64 MB pieces (Tyler's $100 \times 5 \text{ GB}$ shards work too; smaller chunks shrink the tail).

3 API 8–11 min · controller, tracker, peers

- **POST /rollouts** {version, targets} · **GET /rollouts/{id}** → downloaded · verified · active, stalled hosts, ETA
- **agent → tracker:** **POST /hosts/{id}/have** {chunk_id, hash_ok} · **GET /chunks/{id}/peers?rack=** · **POST /hosts/{id}/heartbeat** {bitmap, health}
- **peer → peer:** **GET /chunk/{id}** (range), concurrent-peer cap per host (incast)
- Operators act on **verified + active**, never on "downloaded".

4 DATA FLOW 11–18 min · one chunk, then one host

- controller publishes the manifest → 2–4 seeds pull chunks from the repo (the only traffic the source link carries) → seed verifies chunk 17's hash → tells the tracker "I have 17" →
- a host asks for peers of the chunks it lacks, **rarest first**, several in flight → downloads, verifies, registers as a source for that chunk **while still downloading the rest** →
- **bitmap full** → whole-file hash → temp path → atomic symlink swap → reports verified → readiness gate: only verified, correct-version hosts take traffic.
- **Host dies:** its chunks are on other peers; tracker drops it on missed heartbeats; neighbours re-fetch; the source is the fallback. No human.

5 HIGH-LEVEL DESIGN 18–30 min · three designs in one breath, then the sum

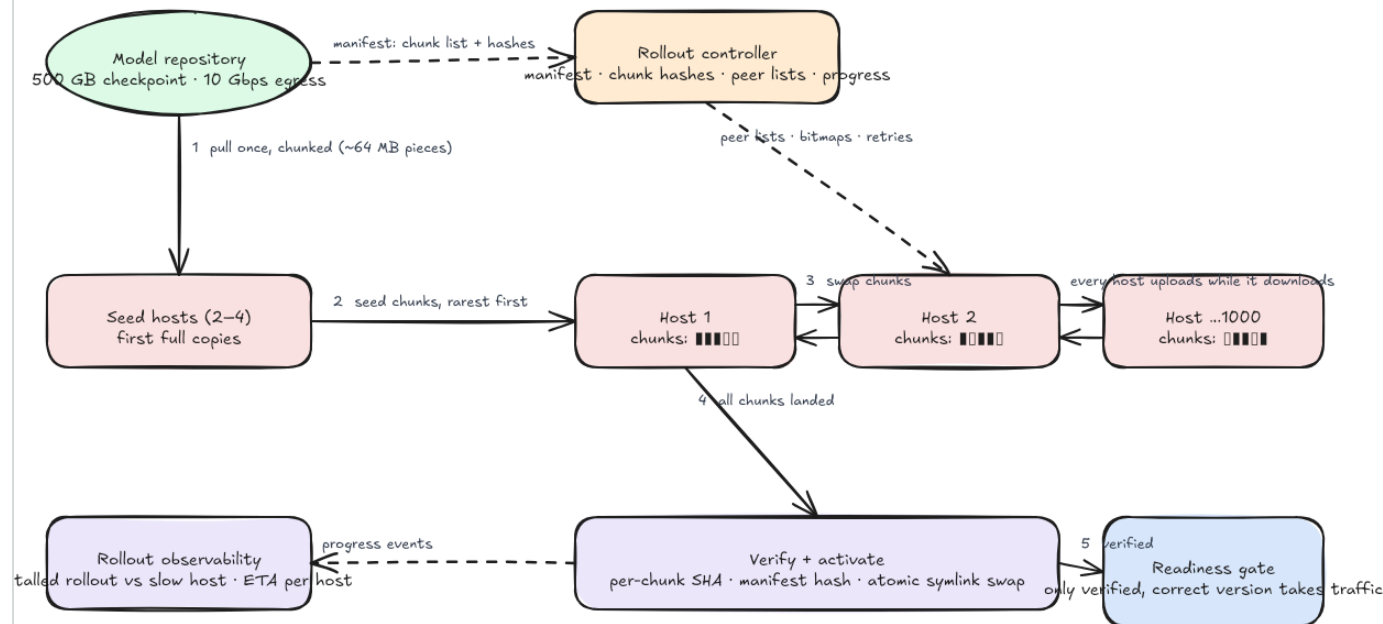
- **Naive:** 1,000 pulls through one 10 Gbit/s link = $1,000 \times 400 \text{ s}$ (days). **Tree:** seeds → levels; better, but each level waits for whole files. **Swarm:** chunked, every host uploads while it downloads; bandwidth adds up, completion → the bound.
- **Board:** repository → seeds → swarm → verify + activate → readiness gate; beside it the controller (manifest, progress), the tracker (chunk maps, bitmaps, health), observability (stalled rollout vs slow host). Solid = bytes, dashed = control.
- **The sum, unprompted:** root splits its NIC 5/5 pull/serve → 800 s to hold every chunk; last 5 GB chunk $\times \sim 11 \text{ hops} \times \sim 4 \text{ s} \rightarrow 44 \text{ s}$; $\approx 844 \text{ s} \approx 14 \text{ min}$ for 1,000 hosts, within $2\times$ of the bound. More seeds or smaller chunks → closer to 400 s.

6 DEEP DIVES 30–45 min · the card's follow-ups

- **Bad or slow host:** hash mismatch → quarantine as a source, re-fetch elsewhere; measure per-peer throughput, replace slow neighbours, backoff; cap concurrent peers (incast).
- **Coordinator-free recovery:** tracker dies → hosts gossip bitmaps, pull rarest-first with no central assignment.
- **Readiness + observability:** verified + correct version or no traffic; per-host completeness + ETA; bandwidth caps and windows.
- **Cadence:** a version every few hours → keep previous for rollback, pre-warm the next, delete the one before; rack-aware peers if there is topology.
- **If it turns into Kyle's variant:** generation-checked reports, stale "done" ignored, 3 regions (3 copies = 240 s, then in-region fan-out), rollback = new generation (see the rollout sheet).
- **Bow:** bound, seed once, swarm, verify, activate atomically, route only verified, tracker replaceable by gossip. Left: incast + root split; validate swarm throughput on one rack.

Model weight distribution to 1,000 hosts

500 GB checkpoint, one 10 Gbps source, 10 Gbps per host. State the bound, then the swarm. Solid = data, dashed = control.



Failure: a host dies mid-rollout → its chunks live on other peers; neighbours re-fetch outstanding pieces (rarest first); source is the fallback. No coordinator needed if hosts gossip bitmaps.
Bad chunk → hash mismatch → quarantine that host as a source, re-fetch elsewhere.

Legend

source / storage control plane hosts (data plane) verification, observability serving gate

Numbers to say out loud, in the first five minutes

Lower bound: each host must receive 500 GB over a 10 Gbps NIC → $500 \times 8 / 10 = 400$ s. Nothing beats that. (800 s if up and down share the link.)

Naive pull from the source: $1,000 \times 400$ s. Tree broadcast: better but each level waits for the whole file. Chunked swarm: bandwidth adds up, completion → the bound.

Cap concurrent peers per host (incast); rack-aware peer choice if there is topology; bandwidth caps so production traffic survives.